

点分治

给定一棵有 n 个点的树，给定 m 个询问树上距离为 k 的点是否存在(或者说有多少对满足条件的点)

输入：

2 1

1 2 2

2

输出：

AYE

$n \leq 10000$, $m \leq 100$, $c \leq 10000$, $K \leq 1000000$

点分治

对于这种求解树上距离的问题，点分治是一种很暴力并且很好用的算法。

点分治的原理很简单，我们先想一下我们以前如何对一个大的区间分治的？

一直二分一直二分，这样就把一个大区间分成了很多小区间，查找的效率很高， $\log n$ 。

那么如果需要把树分治呢？该如何划分呢？

也就是我们先找到一个点，把一个完整的树以这个结点为中心拆分成几棵子树，再去每棵子树里面继续递归拆分。

关键是怎么选点呢？随机选点？那就只能看人品了？选择每棵树的第一个点？那随便就可以出数据卡死。

我们很明显需要一个可以把树比较均分的点。

树的重心

这个点就是树的重心。

树的重心：树上的一个点，其所有的子树中最大的子树的结点数最少，那么这个点就是重心。

因为我们需要递归的对树进行分治，那么肯定分治次数越小越好，那么通过重心分治下来的数就是比较均衡的，效率就高。

树的重心的求法以前就讲过，可以 $O(n)$ 的求出，因为这里需要递归的求树的重心，所以稍微有点变化。

那么分治的大致思路就清楚了，边通过找到的重心对树进行分治，再去分治的树中去找重心，再分治。

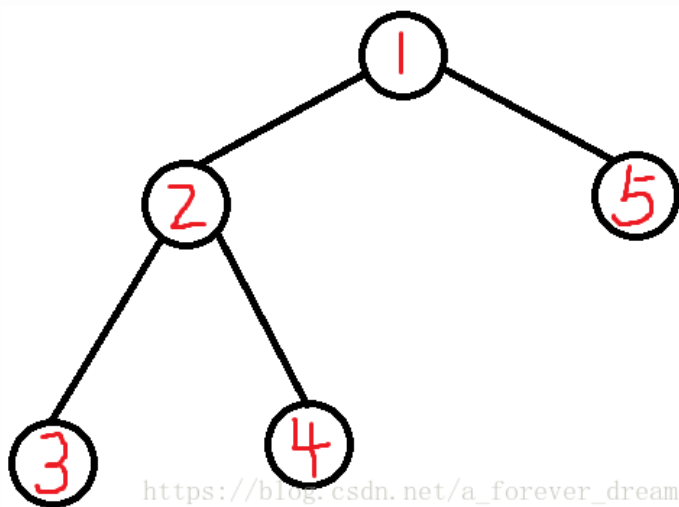
树的重心

```
void getroot(int u,int fa)//求树的重心
{
    size[u]=1;
    int Max=0;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v==fa||visit[v])//visit表示该点是否被分治过
            continue;
        getroot(v,u);
        size[u]+=size[v];
        Max=max(Max,size[v]);
    }
    Max=max(Max,Size-size[u]);//以Size表示当前这棵子树的大小
    if(Max<Ans)
    {
        Ans=Max;
        root=u;//root表示重心
    }
}
```

点分治

当我找到重心之后，因为需要求两点 (x, y) 之间的简单路径，那我们完全可以通过重心把他分成两部分， x 到重心的距离， y 到重心的距离，所以我们这里开一个 dis 数组， dis 数组表示该点到当前子树重心的距离。

那么问题就出现了？并不是所有两点间的简单路径都通过重心啊，没通过的怎么办？



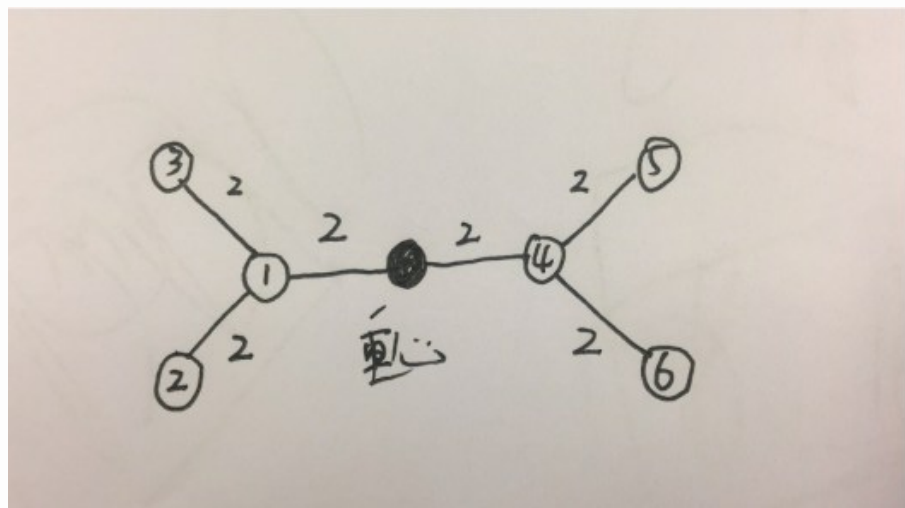
假设1是重心，路径 $3 \rightarrow 4$ 之间的简单路径不通过重心，所以计算是错误的。

点分治

对于这种情况也没有关系，如果两个点的简单路径没有经过当前重心，是没有办法计算的。也就是计算出来的答案是不合理的。即如果路径的两个端点都在以该点为重心所分成的一棵子树内，说明是不合理的，所以减去就可以了。

我们用总的方案数-不合理的方案数就得到了合理的方案数。举一个例子

点分治



以黑点为重心的时候，一棵树被分成左右两棵子树，此时如果要求路径长度为8的可能性，得到的答案是 $4 \times 3 / 2 = 6$ ，明显是错误的，没关系，后面再减。

当我们递归到左子树的时候，假设1是重心，那么以1为重心继续遍历到2, 3，那么得到3，2到总树的重心的路径长度还是4，那么此时就需要减去这种情况， $2 \times 1 / 2 = 1$ ，同样，右边也是一样的原理，最终得到 $6 - 2 = 4$ 。

实际上就是容斥原理，如果以该点为重心时计算过，那么在该点的子树中出现了该情况就应该减去，

点分治

```
void fenzi(int u,int sum)//当前分治的子树之和
{
    visit[u]=1;//标记表示已经分治过了
    solve(u,0,1);//solve表示统计答案, 1表示加上的
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(visit[v])
            continue;
        solve(v,edge[i].w,-1);//-1表示需要减去的, 子树中的要减去
        Ans=inf;//后面要dfs找重心, 所以先初始化
        Size=size[u]<size[v]?sum-size[u]:size[v];
        getroot(v,v);//继续找子树的重心
        fenzi(root,Size);//继续分治子树
    }
}
```

点分治

接下来就是如何计算了，题解的计算方法大多有问题，都是极限 $O(n^2)$ 的去枚举，数据比较水的时候可以过，但是考试明显会被卡爆。

第一步先预处理出来所有的 $dis[i]$ ， i 结点到当前重心的距离。重新开一个数组存起来

```
void getdis(int u, int fa, int len)
{
    dis[++t] = len; // 不需要知道是哪一个点，只需要知道有多少个
    for(int i = head[u]; i; i = edge[i].next)
    {
        int v = edge[i].to;
        if(v == fa || visit[v])
            continue;
        getdis(v, u, len + edge[i].w); // 注意长度要+当前边的长度
    }
}
```

点分治

重新存起来的好处是什么呢？假设我们需要知道经过当前重心的点长度为 k 的点有多少个。第一种办法是 $n*n$ 的枚举，然后break，但是明显容易被卡，我们可以排序然后二分，排序之后去重，统计一下该长度的数量。

```
t=0;tt=0;//tt表示有多少个不同的距离
getdis(u,u,len);//处理该子树的所有路径长度
sort(dis+1,dis+t+1);//排序，二分查找
dis[0]=-1;
for(int i=1;i<=t;i++)
{
    if(dis[i]!=dis[i-1])    arr[++tt].x=dis[i],arr[tt].y=1;
    else                    arr[tt].y++;//y表示数量
}
```

点分治

接下来就是统计答案了。如果要求长度为 k ，我们先枚举 $arr[i].x$ ，再去后面二分查找，如果存在 $arr[i].x + arr[mid].x == k$ ，那么就在以前的答案的基础上累加上 $arr[i].y * arr[mid].y$ 。但是一定要区分开是否在当前重心的根结点内。如果是，就是减去，如果不是，就加上。

但是这样做是有问题的，如果 k 是一个偶数，那么存在 m 个满足条件的点，其中肯定有 $k/2$ 这样的点，那么这种点的个数应该是 $arr[i].y * (arr[i].y - 1) / 2$ 。所以最好要特判，不然会查半天错。

点分治

```
for(int i=1;i<=m;i++)//枚举查询
{
    if(a[i]%2==0)//如果是偶数，特判
    for(int j=1;j<=tt;j++)
    {
        if(arr[j].x==a[i]/2)
            ans[i]+=(arr[j].y-1)*arr[j].y/2*flag; //flag表示是+还是-
    }
    for(int j=1;j<=tt&&arr[j].x<a[i]/2;j++)//前算过的不能再算了
    {
        int l=j+1,r=tt;
        while(l<=r)
        {
            int mid=(l+r)/2;
            if(arr[j].x+arr[mid].x==a[i])//如果刚好满足
            {
                //因为不存在重复，所以当前情况肯定不会再有满足的了
                ans[i]+=arr[j].y*arr[mid].y*flag;
                break;
            }
            if(arr[j].x+arr[mid].x>a[i])//二分
                r=mid-1;
            else
                l=mid+1;
        }
    }
}
```

练习题

点分治主要用于树上路径统计问题的算法。

洛谷 3806 点分治
洛谷 4178 Tree
洛谷 4149 Race
洛谷 2634 聪聪可可
洛谷 3714 树的难题
SP0J 1825 免费旅行
BZOJ 3784 树上的路径
BZOJ 3730 震波



分治

分治我们很早之前就学习过，比如二分查找，二分答案，三分查找，还有一系列和二分有关的数据结构和算法。但是我们以前学习的分治两个部分是相对独立的，即左边部分不会对右边部分产生影响，我们只要的是两个独立的子集合并后的答案。但是很多时候其中一个部分是对另一个部分产生影响的，我们来看一道很水的题





逆序对

求一个序列的逆序对。

这是一道现在看来再简单不过的题了，树状数组的一道裸题，归并排序也可以做。

实际上可以分治。

分治的思想就是把一个大问题分成很多个小问题，直到这个小问题可以直接求解问题，再把这些小问题一点一点的合并成原来的问题

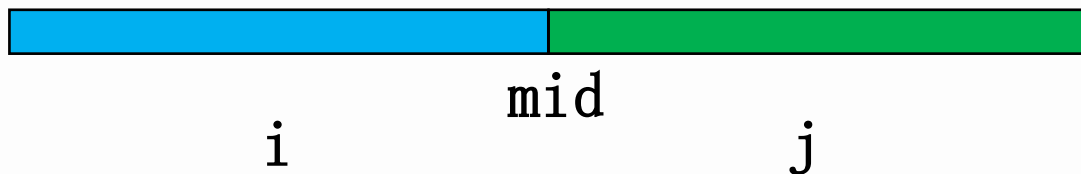


逆序对

如果要求逆序对，分治到什么时候才可以直接直到逆序对的数量呢？当然是数列中只有1个数的时候，那么该数列的逆序对为0.

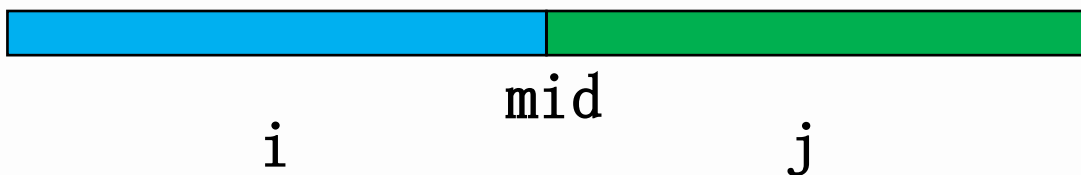
接下来就需要合并了，1个数的时候没有逆序对说明这个序列是有序的，那么在合并两个有序序列的时候如何才能判断逆序对的个数呢？用前一个序列的每一个数和后一个数的每一个比较？太慢了。浪费了有序的性质。

对于两个有序表中的位置 i 和 j ,表示此时已经合并到 i 和 j 这个位置了，如果 $a[j] < a[i]$ ，那么仅表示表示 (i, j) 为逆序对吗？



逆序对

远远不止，我们知道 i 所在序列是有序的，那么在该序列中 i 后面的数都比 i 这个位置的值大，那么如果 $a[i] > a[j]$ ，不就相当于后面的数也都大于 $a[j]$ 吗，那么此时就说明当前整个序列中 j 的逆序对有 $(mid - i + 1)$ 个。



那么当我 $O(n)$ 的合并有序表的时候，就可以不重复不遗漏地算出当前序列中的所有逆序对。

有人会问？合并之后这个序列不是有序的吗？那对算逆序对有影响吗？

不会有，因为当前序列的逆序对已经求完了，在合并的时候，在 j 后面的仍然在 j 后面，在 i 前面的还在 i 前面。

CDQ分治

```
void merge(int l,int r)
{
    if (l==r) return;
    int mid=(l+r)>>1;
    merge(l,mid);
    merge(mid+1,r);
    int t1=l;
    int t2=mid+1;
    for (int i=l;i<=r;i++)
    {
        if ((t1<=mid&& a[t1]<=a[t2])||t2>r) {
            b[i]=a[t1];
            t1++;
        }
        else
        {
            b[i]=a[t2];
            ans+=(mid-t1+1);
            t2++;
        }
    }
    for (int i=l;i<=r;i++) a[i]=b[i];
}
```

这就是分治一个简单模型，在合并的时候，前面子序列会对后面子序列产生影响，而我们需要计算出这个影响，CDQ分治就是用来求解这一类问题的算法。

普通的分治大多只能用来求解两个子区间相互独立的分治算法，而CDQ分治主要用来求解后一个子区间会受到前一个子区间影响的这一类问题。

CDQ分治

```
void merge(int l,int r)
{
    if (l==r) return;
    int mid=(l+r)>>1;
    merge(l,mid);
    merge(mid+1,r);
    int t1=l;
    int t2=mid+1;
    for (int i=l;i<=r;i++)
    {
        if ((t1<=mid&& a[t1]<=a[t2])||t2>r) {
            b[i]=a[t1];
            t1++;
        }
        else
        {
            b[i]=a[t2];
            ans+=(mid-t1+1);
            t2++;
        }
    }
    for (int i=l;i<=r;i++) a[i]=b[i];
}
```

这就是分治一个简单模型，在合并的时候，前面子序列会对后面子序列产生影响，而我们需要计算出这个影响，CDQ分治就是用来求解这一类问题的算法。

普通的分治大多只能用来求解两个子区间相互独立的分治算法，而CDQ分治主要用来求解后一个子区间会受到前一个子区间影响的这一类问题。